

task_bridge

**Votre tableau de tâches et votre source de vérité,
impeccablement synchronisés — dans les deux sens.**

Résumé

task_bridge est un moteur de synchronisation générique, découplé et bidirectionnel de tâches/tableaux intégré à Go. Il maintient en synchronisation déterministe la source de vérité des éléments exploitables SQLite d'un projet avec ses documents de suivi et un tableau distant (première cible : ClickUp ; Jira et Linear prévus), selon une logique de *dernière modification prioritaire*, avec exécution en mode sec d'abord et une garantie de non-corruption des données.

Description courte

Un sous-module de Go, agnostique aux projets, qui synchronise de manière bidirectionnelle une source unique de vérité (SSoT) d'éléments exploitables SQLite ↔ documents de suivi ↔ un tableau distant (ClickUp en premier lieu). Synchronisation déterministe (*dernière modification prioritaire*), exécution en mode sec d'abord, webhooks vérifiés par HMAC ; chaque identifiant et chaque credential est injecté par le consommateur au moment de l'exécution.

Description longue

Toute équipe finit par gérer deux registres d'un même travail : le vrai — code, documentation, base de données interne — et celui que surveillent les responsables, un tableau comme ClickUp. Les deux divergent dès que l'un ou l'autre est modifié, et les réconcilier manuellement relève de ces corvées fastidieuses et sources d'erreurs que personne n'accomplit de manière fiable. task_bridge a été conçu pour éliminer cet écart en traitant les trois représentations comme un seul système maintenu en parfaite synchronisation : la **source unique de vérité des éléments exploitables SQLite** d'un projet, sa **documentation**

de suivi, et un **tableau distant** — le premier tableau pris en charge étant ClickUp, avec Jira et Linear prévus à l’avenir. La synchronisation est déterministe (*dernière modification prioritaire*), s’exécute d’abord en mode sec, et repose sur une promesse non négociable : elle ne corrompra ni ne perdra jamais de données, et ne laissera jamais l’une des parties obsolète sans avertissement. Dans un domaine où une synchronisation négligente peut effacer une semaine de travail, cette approche sécurisée est l’objectif même du module. Sur le plan architectural, il s’agit d’un sous-module strict, intégré à d’autres projets et entièrement agnostique aux projets, conformément au contrat de découplage de la constitution (§11.4.28) : il ne contient aucune valeur spécifique à un projet, et chaque credential, identifiant de tableau/dossier, champ de clé d’élément ou chemin de base de données est injecté par le consommateur au moment de l’exécution via pkg/config.Config. Le module est structuré en couches distinctes : un CLI (reconcile/push/pull/resolve/status/conflicts/init) et un démon en arrière-plan (récepteur de webhooks + réconciliation planifiée) ; un client léger s’appuyant sur la bibliothèque MIT raksul/go-clickup ; un résolveur transformant les URL de tableaux/dossiers en identifiants via des sondes API en temps réel (sans deviner la grammaire URL) ; un mappeur établissant la correspondance entre les éléments exploitables locaux et les champs des tâches distantes ; un moteur de synchronisation *dernière modification prioritaire* avec gestion explicite des conflits ; et un récepteur de webhooks vérifiant la signature X-Signature via HMAC-SHA256. Le module assume son niveau de maturité : il s’agit d’une structure P1 — l’architecture, les interfaces, les points d’entrée et la frontière de découplage sont en place, mais la logique de synchronisation et les appels à ClickUp ne sont pas encore implémentés (chaque stub retourne une erreur explicite *non implémenté*, conformément à la règle *no-fakes*).

Pourquoi nous l’avons développé

Les équipes conservent l’état « réel » du travail dans le code et la documentation, tandis que les responsables évoluent dans un tableau comme ClickUp — et les deux divergent en permanence. task_bridge en fait un seul système, synchronisé de manière déterministe et sécurisée, afin qu’aucune des deux parties ne devienne obsolète ou incorrecte.

Contenu

Pourquoi c'est un changement de paradigme

La synchronisation bidirectionnelle des tableaux est généralement une intégration ponctuelle, figée et codée en dur, que chaque équipe recrée de manière approximative. `task_bridge` la repense comme une bibliothèque réutilisable, injectée par identifiants, avec des garanties strictes de sécurité des données intégrées dès la conception : exécution en mode simulation en premier lieu, résolution déterministe des conflits par la dernière modification, vérification des événements par HMAC. Ainsi, n'importe quel projet peut adopter une intégration fiable des tableaux en injectant simplement une configuration, plutôt qu'en développant une fois de plus un connecteur fragile, étroitement couplé à ses structures internes.

Ce qui est innovant

- Synchronisation bidirectionnelle à trois voies : SQLite SSoT ↔ documents de suivi ↔ tableau distant.
- Découplage total (§11.4.28) : aucune valeur propre au projet ; tout est injecté à l'exécution.
- Résolution Live-API URL→ID au lieu d'un parsing fragile de la grammaire URL.
- Ingestion de webhooks vérifiée par HMAC-SHA256 pour les événements en temps réel.

Défis et solutions

- **Sécurité des données entre trois sources** : résolu grâce à une résolution déterministe des conflits par la dernière modification, une exécution en mode simulation en priorité, et des résultats de conflits explicites.
- **Réutilisabilité sans couplage** : résolu via la frontière d'injection pkg/config (aucune spécificité projet embarquée).
- **Identification fiable des tableaux** : résolu en convertissant les URL en ID via des sondes Live-API.
- **Transparence de l'échafaudage** : résolu en faisant retourner aux stubs non implémentés des erreurs explicites de type *non implémenté* (pas de faux-semblants).

Pile technologique (pourquoi + comment)

- **Go** — moteur, CLI (`cmd/task_bridge`) et démon (`cmd/task_bridged`).

- **SQLite** — la source unique de vérité pour les éléments exploitables.
- **rakul/go-clickup (MIT)** — enveloppe de transport pour ClickUp.
- **HMAC-SHA256** — vérification des signatures des webhooks.
- **cron + webhooks** — réconciliation du démon et ingestion des événements en temps réel.
- **pkg/config** — frontière d'injection des identifiants et des configurations à l'exécution.

Transparence du statut : il s'agit d'un **échafaudage P1** — la logique de synchronisation n'est pas encore implémentée.
Ne pas présenter comme une solution livrée.